

Project Report:

Efficient Text Compression using Huffman Encoding (C++)



AROONA NOOR 01-134242-027

MUSKAN AKASHA 01-134242-102

ZAHRA KHAN 01-134242-139

BSCS-3C

Submitted to: Ms. Ayeza Ghazanfar

Department of Computer Science

BAHRIA UNIVERSITY, ISLAMABAD

Table of Contents

Project Report: Efficient Text Compression using Huffman Encoding (C++).....	1
ABSTRACT.....	3
Introduction.....	4
Project Objectives	4
Theoretical Foundation	4
Design and Implementation Details	4
1. Data Structures Used.....	4
I. Node Structure	4
II. Priority Queue with manual comparator	5
III. Frequency and Code pair	5
2. File Structure.....	5
3. Core Algorithms Used	5
I. Frequency Analysis.....	5
II. Huffman Tree Construction	6
III. Code Generation	8
IV. Text Encoding:.....	8
Results and Statistics.....	9
File I/O Operations:	9
Results:.....	11
Flowchart:	12
Group Contributions	13
Conclusion:	13
Critical Thinking Component	14

ABSTRACT

This project implements **Huffman Encoding**, a lossless data compression algorithm, using C++ to demonstrate efficient text compression through optimal prefix coding. Huffman Coding is an approach to text compression originally developed by David A. Huffman while he was a Ph.D. student, and published in the 1952 paper "A Method for the Construction of Minimum-Redundancy Codes". It generates variable-length codes. The character with least frequency gets the largest code and vice versa. It utilizes Min Heap and does frequency analysis. The implementation features both encoding and decoding functions, file I/O operations, and statistics reporting including compression ratios. This project shows a real life application of Huffman Encoding in the form of text compression and also exemplify the greedy algorithm. Different data structures like binary trees and priority queues are used.

Introduction

Digital data is growing and requires efficient mechanisms for storage and transmission. Uncompressed text files, especially the ones with repeated use of the same character, are extremely bandwidth- and storage-intensive. This project implements one of the most common lossless data compression algorithms known as Huffman Encoding. Our aim is to provide a practical application of the core data structures and algorithms-the Binary Tree and the Priority Queue-to achieve efficient compression, also keeping in view the data integrity.

Project Objectives

- Implementation of the Huffman Encoding algorithm.
- Frequency analysis and building a sorted frequency table.
- Build Huffman Binary Tree using a Min-Heap Priority Queue.
- To generate Codes by DFS tree traversal.
- Demonstrate encoding and decoding.
- Calculate and report the compression ratio and statistics.

Theoretical Foundation

Huffman coding works by:

- Analyzing and counting character frequencies in the input data
- Building an optimal binary tree (Huffman tree) using a greedy algorithm
- Generating variable-length prefix codes where frequent characters get shorter codes
- Ensuring no code is a prefix of another (prefix property)

Design and Implementation Details

The design and implementation of Huffman Encoding is as follows.

1. Data Structures Used

I. Node Structure

```
struct Node {
    char ch;
    int freq;
    Node* left;
    Node* right;
    Node(char c, int f, Node* l = nullptr, Node* r = nullptr)
        : ch(c), freq(f), left(l), right(r) {}

    ~Node() {
        delete left;
```

```
delete right;  
}};
```

II. Priority Queue with manual comparator

```
struct Compare {  
  
    bool operator()(Node* a, Node* b) {  
        return a->freq > b->freq;    }  
};
```

III. Frequency and Code pair

```
struct FreqPair {  
    char ch;  
    int freq;  
};  
struct CodePair {  
    char ch;  
    string code;  
};
```

2. File Structure

- Program is divided into files for manageability:
- HuffmanEncoder.cpp (the implementation file)
- HuffmanEncoder.h (header file for the prototypes and data structures definitions)
- main.cpp (main driver program for testing implementation)
- input.txt (for input text)
- compressed_bits.txt (encoded binary string)
- output_decompressed.txt (decoded output text)
- Huffman_report.txt (statistics report file)

3. Core Algorithms Used

I. Frequency Analysis

First the text is read from the file:

```
string readFile(const string& filename) {  
    ifstream inputFile(filename);  
    if (!inputFile.is_open()) {  
        cerr << "Error: Could not open file " << filename << endl;  
        return "";  
    }
```

```

    }
    string content(((istreambuf_iterator<char>(inputFile)), (istreambuf_iterator<char>()));
    return content;
}

```

Then a **frequency count** is analyzed for every character. After counting, a manual sorting algorithm, **Bubble Sort** is applied to sort the unique characters in descending order based on their frequency.

```

void HuffmanEncoder::sortFrequencies() {
    for (int i = 0; i < freqCount - 1; ++i) {
        for (int j = 0; j < freqCount - i - 1; ++j) {
            // Compare adjacent elements and swap if they are in the wrong order
            if (frequencies[j].freq < frequencies[j + 1].freq) {
                // Manual swap block
                FreqPair temp = frequencies[j];
                frequencies[j] = frequencies[j + 1];
                frequencies[j + 1] = temp;
            }
        }
    }
}

```

II. Huffman Tree Construction

- a) Initialize a Min-Heap Priority Queue with a node for every unique character.
- b) Insert all leaf nodes to priority queue.
- c) Until the queue is empty, repeat:
 - Extract the two nodes with smallest frequency.
 - Create a new internal node using the two smallest nodes as children. And its frequency will be the sum of the two nodes.
 - Insert this node back into the Priority Queue.
- d) The final node is the root of the Huffman tree.

```

void HuffmanEncoder::buildHuffmanTree() {
    priority_queue<Node*, vector<Node*>, Compare> minHeap;
    for (int i = 0; i < freqCount; ++i) {
        if (frequencies[i].freq > 0) {

            minHeap.push(new Node(frequencies[i].ch, frequencies[i].freq));
        }
    }
    while (minHeap.size() > 1) {

        Node* left = minHeap.top();
        minHeap.pop();

        Node* right = minHeap.top();
        minHeap.pop();
    }
}

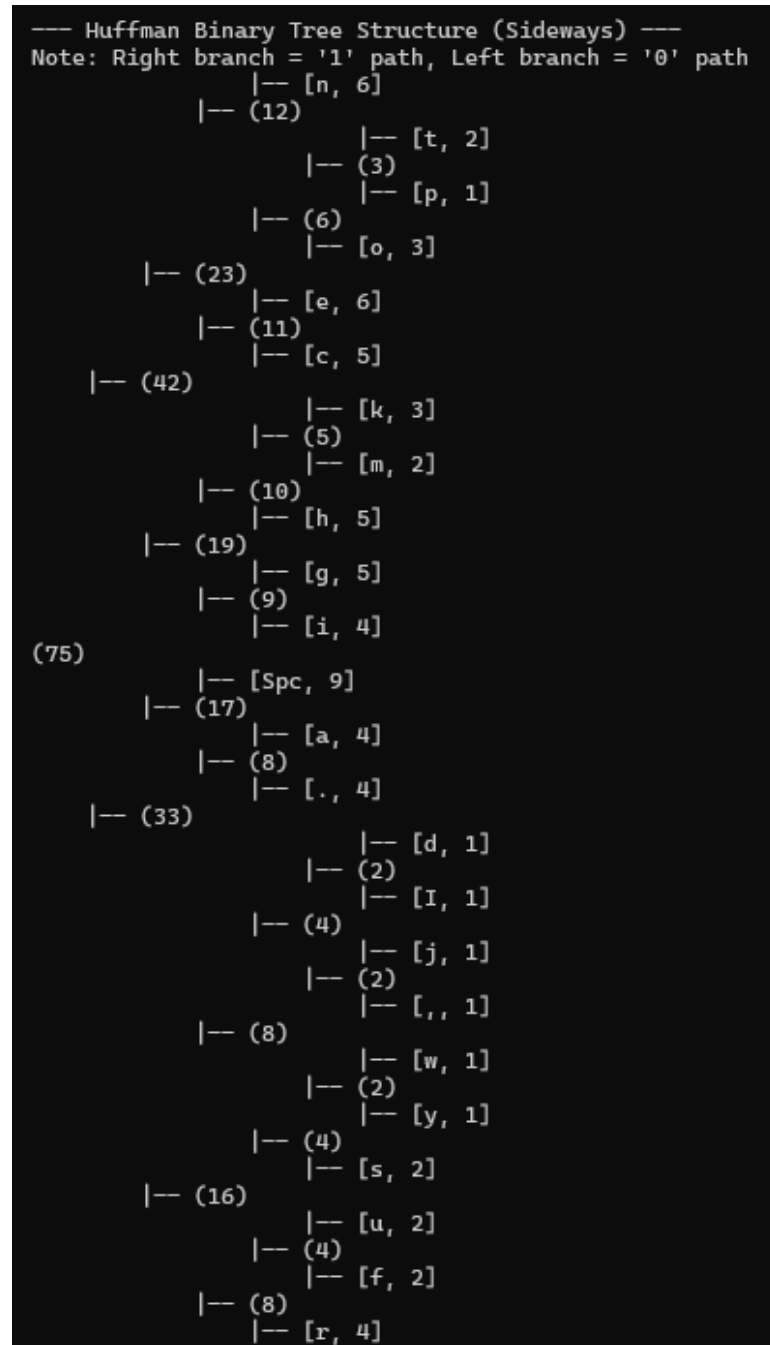
```

```

int sumFreq = left->freq + right->freq;
Node* parent = new Node('\0', sumFreq, left, right);
minHeap.push(parent);
}

```

In the Huffman binary tree structure, the tree is sideways, where the top section is right sub-tree and the root is in middle then the left sub-tree is included.



III. Code Generation

A **Depth-First Search (DFS)** is applied on the binary tree. While moving to the left child appends '0' to the current code, and if moving to the right child appends '1'.

```
void HuffmanEncoder::generateCodes(Node* curr, string code) {

    if (curr->left == nullptr && curr->right == nullptr) {
        codes[codeCount].ch = curr->ch;
        codes[codeCount].code = code;
        codeCount++;
        return;
    }

    // Recursive Step 1: Traverse left (append '0')
    if (curr->left) {
        generateCodes(curr->left, code + "0");
    }
    // Recursive Step 2: Traverse right (append '1')
    if (curr->right) {
        generateCodes(curr->right, code + "1");
    }
}
```

IV. Text Encoding:

When a leaf node is reached, the calculated binary string is assigned as the Huffman Code for that specific character. A binary string of 0 and 1 characters is formed. No code is the prefix of another code.

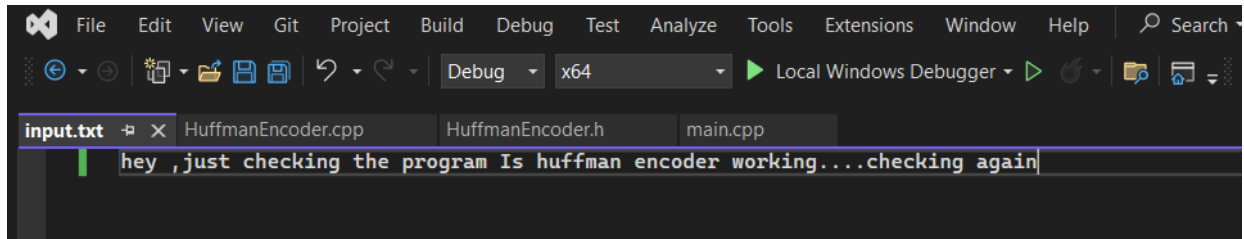
```
string HuffmanEncoder::getCode(char ch) const {
    for (int i = 0; i < codeCount; ++i) {
        if (codes[i].ch == ch) {
            return codes[i].code;
        }
    }
    return "";
}

string HuffmanEncoder::encode(const string& text) {
    if (text.empty()) return "";
    for (char c : text) {
        updateFrequency(c);
    }
    sortFrequencies();
    buildHuffmanTree();
    generateCodes(root, "");
    string encodedBitString = "";
    for (char c : text) {
        encodedBitString += getCode(c);
    }
    return encodedBitString;
}
```


Results and Statistics

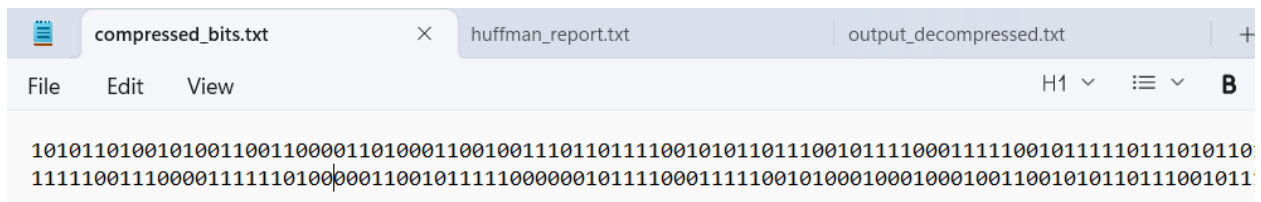
File I/O Operations:

I. Input string in file **input.txt**



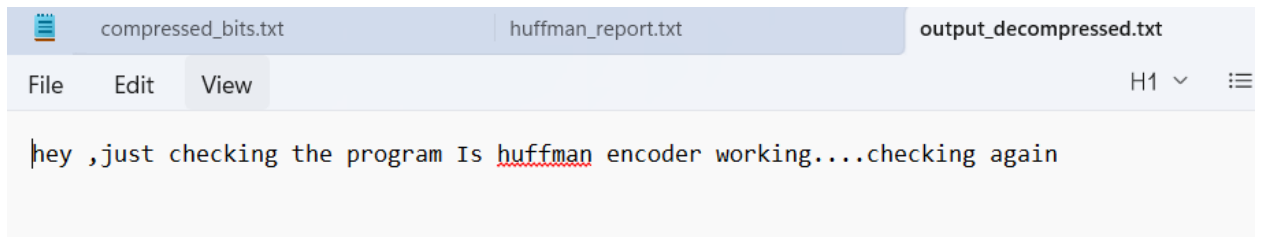
```
File Edit View Git Project Build Debug Test Analyze Tools Extensions Window Help Search
Debug x64 Local Windows Debugger
input.txt HuffmanEncoder.cpp HuffmanEncoder.h main.cpp
hey ,just checking the program Is huffman encoder working....checking again
```

II. Encoded string is written to **compressed_bits.txt**



```
compressed_bits.txt huffman_report.txt output_decompressed.txt
File Edit View H1 B
101011010010100110011000011010001100100111011011110010101101110010111100011111001011110111010110
111110011100001111110100001100101111100000010111100011111001010001000100010011001010110111001011
```

III. Decoded text is written to **output_decompressed.txt**



```
compressed_bits.txt huffman_report.txt output_decompressed.txt
File Edit View H1 B
hey ,just checking the program Is huffman encoder working....checking again
```

- IV. The program provides detailed compression statistics, shown in the output and written to **huffman_report.txt** file.

```

compressed_bits.txt  huffman_report.txt
File Edit View

|
--- Huffman Binary Tree Structure (Sideways) ---
Note: Right branch = '1' path, Left branch = '0' path
      |-- [n, 6]
      |-- (12)
      |   |-- [t, 2]
      |   |-- (3)
      |   |-- [p, 1]
      |   |-- (6)
      |   |-- [o, 3]
      |-- (23)
      |   |-- [e, 6]
      |   |-- (11)
      |   |-- [c, 5]
      |-- (42)
      |   |-- [k, 3]
      |   |-- (5)
      |   |-- [m, 2]
      |   |-- (10)
      |   |-- [h, 5]
      |-- (19)
      |   |-- [g, 5]
      |   |-- (9)
      |   |-- [i, 4]
      (75)
      |   |-- [Spc, 9]
      |-- (17)
      |   |-- [a, 4]
      |   |-- (8)
      |   |-- [., 4]
      |-- (33)
      |   |-- [d, 1]
      |   |-- (2)
      |   |-- [I, 1]
      |   |-- (4)
      |   |-- [j, 1]
      |   |-- (2)
      |   |-- [., 1]
      |-- (8)
      |   |-- [w, 1]
  
```

--- Character Frequencies (Sorted) -
Char | ASCII | Frequency

Spc	32	9
e	101	6
n	110	6
h	104	5
c	99	5
g	103	5
i	105	4
r	114	4
a	97	4
.	46	4
k	107	3
o	111	3
u	117	2
s	115	2
t	116	2
m	109	2
f	102	2
y	121	1
,	44	1
j	106	1
p	112	1
I	73	1
d	100	1
w	119	1

--- Huffman Codes (The Dictionary) ---
Char | Code

r	0000
f	00010
u	00011
s	00100
y	001010
w	001011
,	001100
j	001101
I	001110
d	001111
.	0100
a	0101
Spc	011
i	1000
g	1001
h	1010
m	10110
k	10111
c	1100
e	1101
o	11100
p	111010
t	111011
n	1111

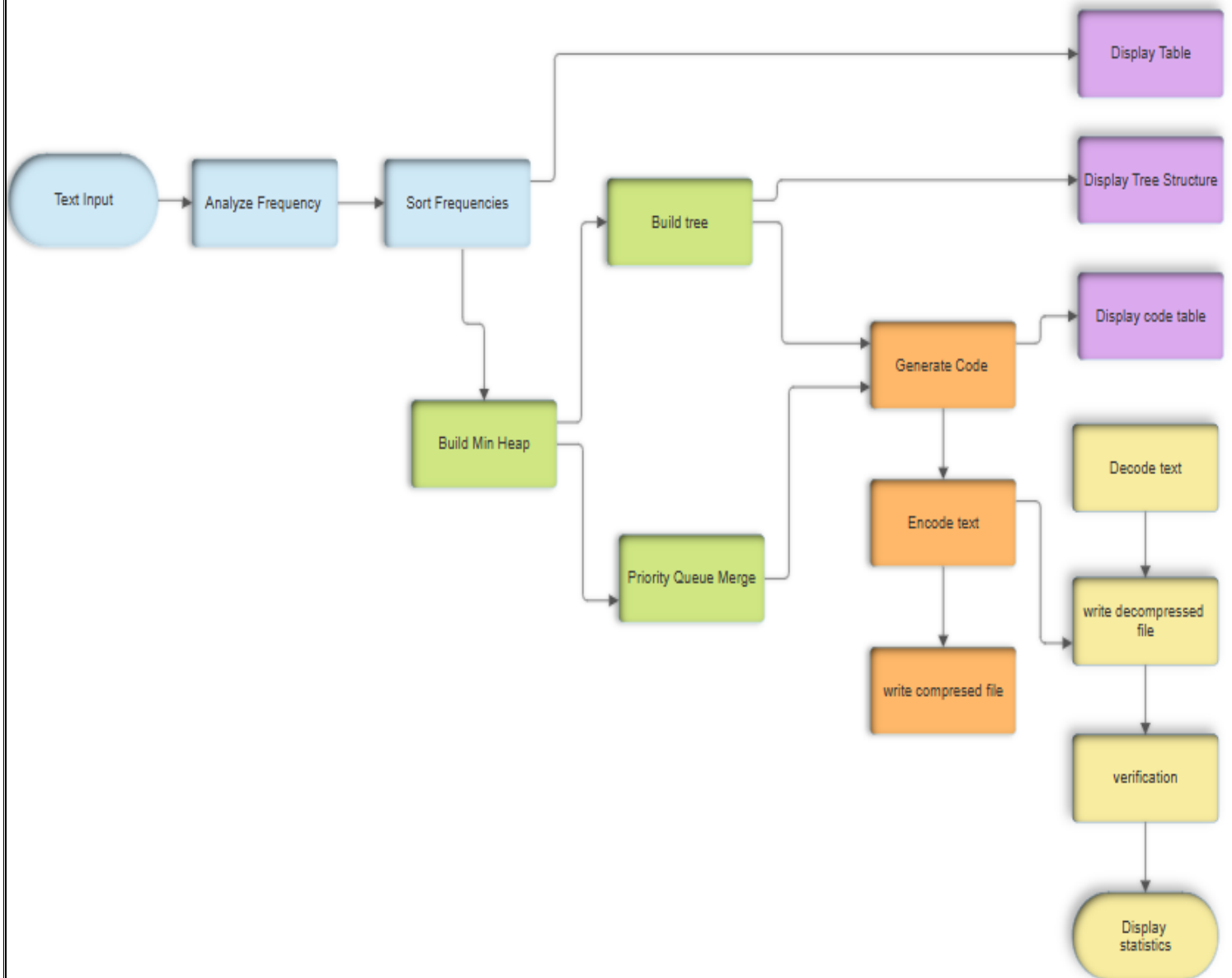
--- Compression Statistics (Task 8) ---
Original Character Count: 75 chars
Original File Size (8-bit ASCII): 600 bits
Compressed Bit Stream Length: 323 bits
Estimated Compressed Size (Bytes): 41 bytes
Compression Ratio (Reduction): 46.17 %

Results:

- **Lossless compression:** Decoded string matches the original string
- **Visualization:** Tree structure display
- **Statistics:** Compression ratio calculation
- **Compression Efficiency:**
Original File Size (Bits) = Original Length (Characters)* 8 bits/char
Compression Ratio = (1 - compressed_bits / (original_chars * 8)) * 100%

```
--- Verification and Statistics ---  
VERIFICATION SUCCESSFUL: Lossless compression confirmed.  
  
--- Compression Statistics (Task 8) ---  
Original Character Count: 75 chars  
Original File Size (8-bit ASCII): 600 bits  
Compressed Bit Stream Length: 323 bits  
Estimated Compressed Size (Bytes): 41 bytes  
Compression Ratio (Reduction): 46.17 %  
-----  
Full report log written to: huffman_report.txt
```

Flowchart:



Group Contributions

Core Algorithms & Architecture by Aroona Noor

- Huffman tree construction
- Priority queue implementation
- Special character handling

User Interface & I/O by Muskan Akasha

- Main driver program
- File read/write operations
- Frequency analysis module
- Documentation

Encoding/Decoding & Statistics by Zahra Khan

- Encoding/decoding algorithms
- Compression statistics
- Verification system
- Debugging

Conclusion:

Huffman Encoding is the optimal data compression method which reduces the size of the original data, removes redundant patterns, produce the shortest codes for the most frequent character. In this project the implementation is done with Binary trees and Priority Queue.

Critical Thinking Component

Can we further compress a file that has already been compressed using Huffman Encoding?

Students are invited to think critically about the following:

1. Is double compression possible?

- What happens if we apply Huffman Encoding again to a Huffman-encoded file?
- Would the file become smaller, remain the same, or increase in size?

Answer:

Although, it is possible to apply Huffman Encoding on an already encoded file, it is not necessarily useful as it usually increases the file size, making the attempt pointless.

After applying Huffman encoding for the first time, the bitstream is at optimal compression, and the resulting data has very low redundancy. That is why, when applying Huffman, a second time while assigning codes to each bit, a small overhead for the new code table will appear, making the encoded output longer. So, the file size will remain almost the same if the probability of 0 and 1 are exactly equal but will slightly increase in the majority of cases because storing the new Huffman tree (for 0 and 1) adds a few bytes of overhead without meaningful compression benefit. Thus, double Huffman encoding does not reduce size further and usually slightly expands the file, making it pointless for compression.

2. Why is additional compression usually not effective?

- Does the encoded output still contain redundancy?
- How does entropy relate to compression limits?

Answer:

Compression is a process of removing redundancies, such as symbols or patterns that occur more frequently. After the first Huffman encoding, the bitstream is already near optimal compression and behaves like a high-entropy data, meaning there is almost no redundancy left. Entropy sets the theoretical limit on how much data can be compressed, and once this limit is reached, further lossless compression is nearly impossible.

That is why, when Huffman encoding is applied a second time, it cannot effectively reduce the size. Instead, storing a new Huffman tree adds a small overhead, which often makes the file slightly larger. If the probabilities of 0 and 1 are exactly equal, the size stays nearly the same, but in most cases, the file expands. Therefore, double Huffman encoding does not reduce size further and is generally pointless.

3. Are there exceptions?

- Are there rare cases where a second compression stage might still reduce size?
- How do multi-stage compression methods (e.g., ZIP, PNG) work?

Answer:

Although rare, a second compression phase might slightly reduce size if the Huffman-encoded data still contains noticeable patterns, such as long runs of 0s or 1s. This could occur if the first encoding did not fully flatten the bit distribution—though such cases are unusual in practice.

More commonly, effective compression systems use multi-stage methods, combining different algorithms to target various types of redundancy:

1. Redundancy Removal: First, patterns or structures are exploited (e.g., LZ77 sliding window in ZIP, filtering + prediction in PNG).

2. Entropy Encoding: Then, Huffman (or Arithmetic) coding is applied to the transformed data, compressing it further.

In these layered approaches, each stage addresses a different form of redundancy—structural, repetitive, or statistical—enabling greater overall compression than a single Huffman pass alone. This layered strategy is why formats like ZIP and PNG are effective, whereas applying Huffman encoding twice in a row generally is not.

4. Real-world use cases:

Students can explore:

- Layered compression (Huffman + LZ77, Arithmetic Coding, Deflate method)
- Compression pipelines in multimedia formats (JPEG, MP3, PNG)
- Scenarios involving nested or embedded compressed data

Answer:

Layered Compression Methods:

These combine different algorithms to achieve better results than any single method alone. The **Deflate method** (used in ZIP, PNG, and GZIP) is a classic example:

It first uses **LZ77** to find and eliminate repeated sequences of data, then applies Huffman coding to compress the resulting symbol stream further.

Similarly, **Arithmetic Coding** is often paired with modelling stages (like prediction or context modelling) for higher compression ratios in formats like JPEG 2000.

Compression Pipelines in Multimedia Formats:

Most image, audio, and video codecs use a multi-step pipeline. For instance:

JPEG: First transforms spatial data (DCT), quantizes it (lossy step), and then applies Huffman or Arithmetic coding to the remaining coefficients.

Transform (DCT) → Quantization → Huffman or arithmetic coding.

MP3: Uses psychoacoustic modelling to discard inaudible sounds (lossy step), followed by Huffman coding of the spectral data.

(psychoacoustic modelling) → frequency transform → Huffman coding.

PNG: Applies a filter to remove pixel correlations, compresses the result with LZ77 (Deflate), and finishes with Huffman coding.

Filtering → LZ77 → Huffman coding

Nested or Embedded Compressed Data: There are scenarios where already-compressed data is embedded within another file. For example, a PDF might contain a JPEG image, or a DOCX file (which is a ZIP archive) might contain compressed XML. In these cases, applying a general compressor (like ZIP) to the entire file yields little benefit, as the embedded data is already compressed. This highlights why understanding compression stages is crucial—compressing data that is already near its entropy limit is ineffective.